Republic of Tunisia

Ministry of Higher Education

and Scientific Research

Higher Private School of Engineering and Technology

TEK-UP

Final-Year Project Report

Submitted in fulfilment of the requirements for the

National Engineering Diploma in Applied Sciences and Technology

Speciality: Software Engineering

Prepared by

Nadim Jebali

Design and Development of a Multi-Vendor E-Commerce Marketplace

Professional Supervisor:	**Mr. Mohammed Aziz Chibani**	Software Engineer
Academic Supervisor:	**Ms. Sawssen Jalel**	Lecturer

Academic Year 2025 - 2026

I authorise the student to submit the internship report for a defence.

Professional Supervisor, **Mr. Mohammed Aziz Chibani**

Signature and stamp

I authorise the student to submit the internship report for a defence.

Academic Supervisor, **Ms. Sawssen Jalel**

Signature

Dédicace

Remerciements

Contents

List of Figures

List of Tables

List of Abbreviations

- **AI** = Artificial Intelligence
- **API** = Application Programming Interface
- **CD** = Continuous Deployment
- **CI** = Continuous Integration
- **CI/CD** = Continuous Integration / Continuous Deployment
- **CRUD** = Create, Read, Update, Delete
- **DevOps** = Development and Operations
- **HTTP** = HyperText Transfer Protocol
- **HTTPS** = HyperText Transfer Protocol Secure
- **IaC** = Infrastructure as Code
- **JSON** = JavaScript Object Notation
- **JWT** = JSON Web Token
- **LLM** = Large Language Model
- **MVC** = Model-View-Controller
- **MoSCoW** = Must have, Should have, Could have, Won't have
- **ORM** = Object-Relational Mapping
- **PFE** = Projet de Fin d'Études
- **RBAC** = Role-Based Access Control
- **REST** = Representational State Transfer
- **SPA** = Single-Page Application
- **SQL** = Structured Query Language
- **UML** = Unified Modeling Language
- **URL** = Uniform Resource Locator
- **WYSIWYG** What You See Is What You Get

General Introduction

Online retail has become a central channel of commerce, yet in Tunisia the digital marketplace landscape remains fragmented. Sellers who wish to reach customers online are largely confined to two unsatisfying options: vertically integrated single-vendor stores, which they cannot join as independent merchants, and classified-ad sites, which list offers but provide no order management, no quality control and no buyer protection. The result is a market with weak tooling for content moderation, product discovery and trust — precisely the qualities that modern marketplaces are expected to guarantee.

It is in this context that the present project was carried out, as a final-year engineering project within the company **NeuralBey**. Its aim is to design and develop a **multi-vendor e-commerce marketplace**: a platform on which independent sellers open and operate their own stores within a shared environment governed by a central operator. The platform lets sellers onboard with minimal friction, publish products and fulfil orders, while administrators review seller applications, moderate content and curate featured selections. Its distinguishing capability is an **artificial-intelligence** verification pipeline that automatically screens stores and products to preserve buyer trust at scale, complemented by a conversational database assistant for operators and an AI-assisted storefront redesign for sellers. The system is engineered with a modern, fully containerised technical foundation and an infrastructure-as-code delivery pipeline.

To conduct the work in an organised and incremental manner, the project adopted the agile **Scrum** method. This report is structured into four chapters. The **first** presents the context and objectives of the project: the host organisation, the study of existing solutions, the proposed solution, the project-management methodology and the modelling language. The **second** analyses and specifies the requirements — actors, functional and non-functional requirements, use-case and class diagrams, the Scrum project backlog and sprint planning, the schedule, and the architecture and technical choices. The **third** and **fourth** chapters describe the realisation of the two successive releases, together with the tests and the deployment. A general conclusion finally draws the assessment of the work and its perspectives.

Chapter 1

Project Context and Objectives

Plan

Introduction

This first chapter lays the foundations of the project. We begin by presenting the host organisation within which the internship took place. We then set out the general context of the project through a study of the existing solutions, a critique of their limitations, and a description of the solution we propose. We continue with the project-management methodology adopted throughout the work, and we close with the modelling language used to specify the system.

1.1 Presentation of the Host Organisation

1.1.1 Company Overview

NeuralBey is a Tunisian technology company that delivers end-to-end engineering services across a broad spectrum of modern computing fields. The company positions itself as a partner that translates a client's vision into a working system, summarised by its own statement: *"From web and mobile apps to AI systems, IoT, embedded engineering, and 3D solutions — we build the technology that powers your vision."*

This breadth allows the company to take a project from initial concept through to deployment without external dependencies, and to combine disciplines that are usually siloed — for example, integrating artificial intelligence into conventional web platforms, or interfacing embedded devices with cloud services. Its portfolio reflects this versatility:

- **Web and mobile applications** for consumer products and enterprise tooling;
- **AI systems**, including model integration, automation pipelines and intelligent assistants;
- **IoT and embedded engineering**, where the company designs firmware and connected devices;
- **3D solutions** for product visualisation, AR/VR and digital twins.

Figure 1.1 shows the official logo of NeuralBey.

Figure 1.1: Logo of NeuralBey

1.1.2 Mission and Vision

NeuralBey operates as a multidisciplinary engineering studio. Its mission is to design and build reliable, modern software and connected systems, and to accompany each project from scoping through to delivery. Every project is staffed by engineers selected from the relevant domain (frontend, backend, AI, hardware, 3D), supported by a professional supervisor who follows the work throughout.

The company's vision rests on three principles:

- **Engineering excellence** — a working culture built on code review, iterative delivery and frequent stakeholder feedback, so that quality is continuous rather than verified only at the end.
- **Cross-disciplinary innovation** — combining fields that are usually separated, such as embedding artificial intelligence into ordinary web and mobile products.
- **End-to-end ownership** — delivering a project from concept to production, including the modern DevOps practices that keep a system maintainable beyond its first release.

These principles aligned naturally with the requirements of a final-year engineering project and shaped the way the present work was carried out.

1.1.3 Organigramme

Figure 1.2 illustrates the organisational structure of NeuralBey, built around a central engineering function organised by technical domain, a project-supervision and quality function, and a research-and-development unit.

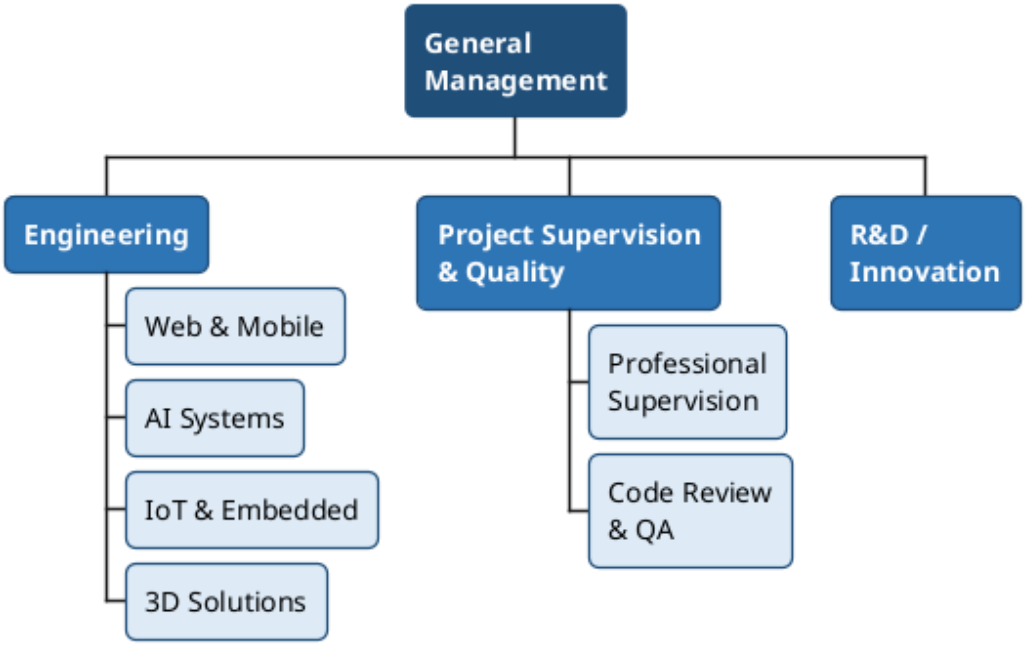

Figure 1.2: Organisational chart of NeuralBey

1.2 Project Presentation

The project originated from an internal need identified at NeuralBey: the Tunisian online-retail landscape lacks a modern, multi-vendor platform that is both seller-friendly and equipped with the automation features expected of contemporary marketplaces. Existing local solutions are typically single-vendor stores or classified-ad sites, and they offer limited tooling for moderation, content quality and discovery.

A *multi-vendor marketplace* is a platform on which independent sellers operate their own stores within a shared environment governed by a central operator. It differs from a **single-vendor store**, where the platform itself owns the inventory and the brand, and from a **classified-ad site**, where the platform merely lists offers and plays no part in transactions or quality control. A marketplace must therefore reconcile three concerns simultaneously: **seller autonomy** (each seller manages catalogue, branding and inventory), **buyer trust** (the platform guarantees a baseline of product authenticity and fulfilment) and **operator governance** (administrators onboard sellers, moderate content, suspend stores and curate featured selections).

Accordingly, three categories of target users were identified at the scoping stage:

- **Customer** — an end-user who browses the catalogue, places orders and tracks deliveries, expecting a fast, mobile-friendly experience with reliable search.
- **Seller** — a small or medium merchant who creates a store, manages a product catalogue and fulfils

orders, expecting low onboarding friction and protection against unfair competition from low-quality listings.

- **Administrator** — the platform operator who reviews seller applications, moderates flagged content, curates featured stores, and intervenes when a store violates the marketplace rules.

1.2.1 Study of Existing Solutions

To position the proposed solution, three categories of competing platforms were studied:

- **Pan-African marketplaces (Jumia).** Jumia Tunisia is the most visible regional marketplace. It supports multiple vendors at scale and offers a polished customer experience, but seller onboarding is heavyweight and the platform is not tailored to small Tunisian merchants.
- **Classified-ad sites (Tayara, Affariyet).** Tayara dominates the local classified-ad market. It offers excellent discovery for sellers, but performs no escrow, no order management, no integrated payment and no quality moderation.
- **Single-vendor stores (MyTek, Wiki, etc.).** These are vertically integrated retailers that operate their own catalogue and cannot be used by independent sellers.

1.2.2 Critique of Existing Solutions

From the study above, four gaps emerged:

L1. No AI-assisted moderation. On existing platforms, store and product verification are entirely manual — slow, inconsistent, and impractical at scale.

L2. Heavyweight seller onboarding. Pan-regional players require contractual paperwork unsuitable for small merchants.

L3. Limited operator tooling. Classified-ad sites have weak admin dashboards; single-vendor stores have none for external sellers.

L4. Outdated technical foundations. Several platforms still rely on monolithic stacks without modern CI/CD or infrastructure-as-code, making evolution costly.

Table 1.1 summarises the comparison along the five criteria that drove the design of the proposed solution. A check mark (✓) indicates that the criterion is fully supported, a cross (✗) that it is absent, and a tilde (∼) partial support.

Table 1.1: Comparative analysis of competing platforms

Criterion	Jumia	Tayara	Proposed Solution
Multi-vendor model	✓	∼	✓
AI verification	✗	✗	✓
Admin dashboard	∼	✗	✓
CI/CD pipeline (open)	✗	✗	✓
Lightweight onboarding	✗	✓	✓

1.2.3 Proposed Solution

The proposed platform is a multi-vendor marketplace designed around three pillars: a self-service seller experience, AI-assisted content verification, and a modern, fully containerised deployment. It is built as a single-page **React** application backed by a modular **NestJS** API, with **MariaDB** as the relational store and **n8n** acting as an orchestration layer for AI tasks. Authentication is handled by **Better-Auth** with email verification, and **Prisma** is used as the ORM. The full system is packaged with Docker Compose, provisioned on DigitalOcean via Terraform, and delivered through a GitHub Actions CI/CD pipeline.

Seven differentiators set the platform apart from the solutions surveyed above:

D1. AI verification of stores and products via an n8n workflow that calls a hosted large-language model and writes back a review flag together with a textual rationale.

D2. Lightweight seller onboarding through a seller application form reviewed by administrators, with a free tier offering a single store at zero cost.

D3. Integrated administration tooling (review, suspend, curate, cascade-delete) implemented as first-class features rather than database admin scripts.

D4. Conversational data access via a database-aware chatbot built on n8n, allowing administrators to query the platform in natural language.

D5. AI-assisted storefront redesign — sellers can regenerate their entire store visual identity from a natural-language prompt, powered by an n8n workflow and a hosted large-language model.

D6. Store-wide sale promotions that propagate a percentage discount automatically across product cards, store listings and search results, applying the discounted price at checkout without per-product editing.

D7. Modern DevOps foundations — a pnpm monorepo, Docker Compose, Terraform on DigitalOcean and GitHub Actions CI/CD — demonstrating production-grade engineering practices end to end.

The scope was agreed with the professional supervisor and bounded to keep the deliverables clear:

In scope: account creation and email verification; the seller application workflow with admin review; store and product CRUD with image upload; tag-based product recommendations with a top-seller fallback; order placement, items and history; admin moderation panels (store suspension, cascade-delete); AI verification and chat assistant via n8n; store-wide percentage sales with automatic badge propagation; AI-assisted storefront redesign; a premium tier unlocking additional stores; and infrastructure-as-code deployment.

Out of scope (deferred to future iterations): integrated payment processing; a native mobile application; advanced fraud detection; on-premise LLM hosting; multi-currency support; and a full-text search engine.

1.3 Project-Management Methodology

Every software project requires a methodological framework to organise work, manage priorities and guarantee the delivery of a quality product. This section justifies the choice of an agile method — **Scrum** — and presents its process, roles, events and artefacts.

1.3.1 Classical vs Agile Approaches

Two major families of methods have historically competed: the **classical** (predictive) approach and the **agile** (adaptive) approach. The classical *Waterfall* model divides the project into strictly sequential phases — requirements, design, development, testing, delivery — each fully validated before the next begins. It suits projects whose requirements are stable and known in advance, but it shows its limits as soon as the context evolves: requirements are frozen at launch, the client sees a working product only at the very end, and defects are detected too late to address without significant impact.

Agility, born from the *Agile Manifesto* (2001) [1], champions collaboration, adaptability and the continuous delivery of value. Rather than planning the entire project upfront, work is organised into **short iterations**, each producing a functional and tested increment. Requirements are refined continuously from feedback, value is delivered from the first iterations, and risks are identified and addressed early.

1.3.2 Main Types of Agile Methodologies

Several frameworks implement the agile principles, each with a different emphasis:

- **Scrum** — organises work into time-boxed *sprints* around three accountabilities and a small set of events; the most widely adopted agile framework.
- **Kanban** — visualises the flow of work on a board and limits work-in-progress, favouring continuous flow over fixed iterations.
- **Extreme Programming (XP)** — focuses on engineering practices such as pair programming, test-driven development and continuous integration.

1.3.3 Why Scrum?

Scrum [2] was selected because it offers the clearest framework for delivering a substantial product in successive, demonstrable increments while keeping requirements open to change. Its sprint rhythm gives the project a regular cadence of planning and review, its backlog provides a single prioritised source of truth for the work, and its review events give the professional supervisor — acting as the stakeholder — frequent opportunities to steer the product. These properties fit a project whose scope was expected to evolve through the internship.

1.3.4 The Scrum Process

Scrum builds the product through a series of **sprints**: short, fixed-length iterations at the end of which a potentially shippable increment is delivered. Each sprint draws its work from the *Product Backlog* into a *Sprint Backlog*, the work is built and tested during the sprint, and the resulting increment is reviewed with the stakeholder before the next sprint begins. Figure 1.3 illustrates this cycle.

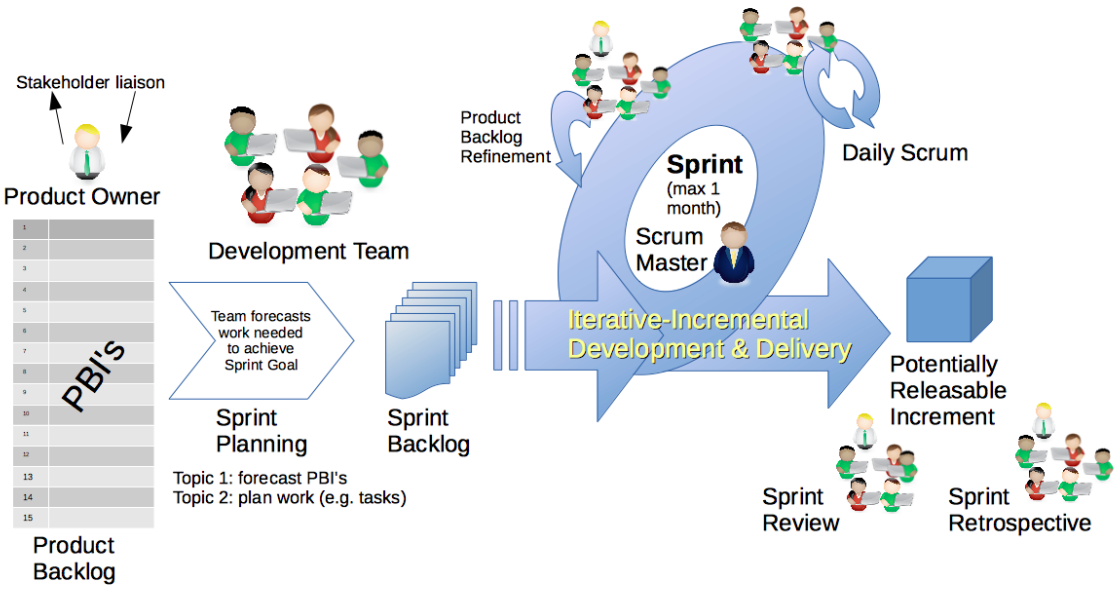

Figure 1.3: The Scrum framework: from the Product Backlog to a shippable increment

1.3.5 Scrum Roles

Scrum defines three accountabilities, mapped to the present project as follows:

- **Product Owner** — owns the product vision and prioritises the backlog. This role was held by the professional supervisor, **Mr. Mohammed Aziz Chibani** (NeuralBey), who also provided architectural guidance.
- **Scrum Master** — ensures the framework is applied and removes impediments. In a single-developer context, this role was assumed by the student.
- **Development Team** — builds the increment. The entire technical realisation — analysis, design, implementation, testing and deployment — was carried out by the student, **Nadim Jebali**.

1.3.6 Scrum Events

The work was punctuated by Scrum's events:

- **Sprint Planning** — selecting, at the start of each sprint, the backlog items to be delivered.
- **Daily Scrum** — a short daily checkpoint to track progress and surface impediments.
- **Sprint Review** — demonstrating the increment to the Product Owner and gathering feedback.
- **Sprint Retrospective** — reflecting on the process to improve the next sprint.

1.3.7 Scrum Artefacts

Three artefacts ensured transparency of the work:

- **Product Backlog** — a prioritised list of all expected functionalities, maintained as user stories with MoSCoW priorities and complexity estimates (detailed in Chapter 1.4).
- **Sprint Backlog** — the subset of backlog items committed to the current sprint.
- **Increment** — a working, demonstrable version of the product delivered at the end of each sprint.

1.4 Modelling Language

To analyse and design the system, the project relies on the **Unified Modeling Language** (UML) [3], the standard graphical notation for specifying and visualising the artefacts of a software system. UML offers complementary diagrams that capture both the structural and the behavioural views of a system: *use-case diagrams* express the functional requirements from the actors' perspective, *class diagrams* describe the static domain model, and *sequence* and *activity diagrams* depict dynamic behaviour. The next chapter uses these diagrams to formalise the requirements and the design of the platform.

Conclusion

This chapter established the foundations of the project. It introduced NeuralBey, the host organisation, and the business motivation behind building a modern Tunisian multi-vendor marketplace. A study of existing platforms revealed four gaps — absent AI moderation, heavyweight onboarding, weak admin tooling and outdated technical foundations — that the proposed solution directly addresses through its key differentiators and bounded scope. Finally, the Scrum methodology and the UML modelling language that frame the rest of the work were presented. The following chapter formalises the requirements derived from this context.

Chapter 2

Requirements Analysis and Specification

Plan

Introduction

This chapter identifies the three system actors, formalises the functional and non-functional requirements of the platform, and illustrates them with UML use-case and class diagrams. It then presents the Scrum management of the project — the team, the product backlog and the sprint planning — followed by the schedule and the architecture and technical choices that underpin the realisation.

2.1 Requirements Specification

2.1.1 Identification of Actors

The platform recognises three actor categories. They are distinct *roles* stored on the `User` table rather than distinct user records: a single account can be promoted from *customer* to *seller* once a seller application is approved, and an administrator account is created automatically on first boot.

- **Customer.** The default role assigned upon registration. A Customer can browse the catalogue, search products, view individual stores, manage a cart, place an order across multiple sellers in a single checkout, and review their order history. Any Customer can submit a seller application and, once approved, gain the Seller role on the same account. Anonymous visitors share the read-only abilities of the Customer but cannot maintain a cart or place an order.
- **Seller.** A Customer whose seller application has been approved by an administrator. A Seller can create and edit one store under the free tier (or several under the premium tier), populate it with products, upload product images, attach product tags, choose a visual template, and consult the orders specific to their store. Sellers cannot review or moderate the work of other sellers.
- **Administrator.** Created automatically by the backend at first boot and not exposed through registration. The Administrator reviews seller applications, browses every store and product, suspends or deletes stores (with cascading deletion), overrides automated AI verification decisions, curates featured content, manages categories, and consults the platform through a database-aware AI chat assistant.

2.1.2 Functional Requirements

The functional requirements are grouped into seven domains. Each domain corresponds to a backend NestJS module and to a matching set of pages on the React frontend.

User Registration and Authentication.

FR-A1. The system shall allow a visitor to create an account with an email address and password.

FR-A2. The system shall send a verification email containing a one-time link upon successful registration.

FR-A3. The system shall prevent unverified accounts from creating a store or placing an order.

FR-A4. The system shall authenticate returning users via Better-Auth sessions and issue an HTTP-only cookie.

FR-A5. The system shall allow the user to log out and invalidate the corresponding session.

FR-A6. The system shall enforce role-based access control on every non-public route through guards and a `@Roles()` decorator.

Store Management.

FR-S1. A Seller shall create a store with a name, description, logo and template, and edit it at any time.

FR-S2. A free-tier Seller shall be limited to a single store; a premium-tier Seller shall operate multiple stores.

FR-S3. Each store shall expose a public storefront URL identified by a human-readable slug derived from its name, with uniqueness enforced and the slug regenerated when the name changes.

FR-S4. The Seller shall customise the storefront across seven independent sections (theme, announcement bar, banner, header, product grid, sidebar, footer) through a WYSIWYG visual editor, with undo/redo and selectable presets.

FR-S5. Customisation features shall be tiered: free accounts use a curated subset, while premium accounts unlock the full set; on premium expiry, existing customisation is preserved but no longer editable.

FR-S6. A Seller shall activate a store-wide percentage sale that propagates a discounted price and a sale badge across product cards, the store card and search results, and applies at checkout.

FR-S7. A Seller shall trigger an AI-assisted storefront redesign by submitting a natural-language prompt, which generates a complete customisation applied live to the editor for review.

Product Management.

FR-P1. A Seller shall create, edit and delete products in their own store(s).

FR-P2. Each product shall accept one or more images of validated type and size, and an arbitrary number of tags used by the recommendation engine.

FR-P3. Each product shall carry an `aiReviewed` flag set by the verification pipeline and an `adminReviewed` flag set on manual override.

FR-P4. Products shall be browsable by category and sub-category.

Order System.

FR-O1. A Customer shall add and remove items in a client-side cart.

FR-O2. Checkout shall create a single order even when the cart spans multiple stores, with one order item per product line, and trigger a confirmation email.

FR-O3. Each Customer shall consult their order history and each Seller the orders that include at least one of their products.

Administration.

FR-D1. The Administrator shall access a dashboard listing pending seller applications, recent activity and platform statistics.

FR-D2. The Administrator shall approve or reject seller applications, granting the Seller role on approval.

FR-D3. The Administrator shall suspend, reactivate or delete any store, with cascading deletion of its products and orders, and override any AI verdict.

AI Verification.

FR-V1. Once its images are uploaded, every store and product shall be submitted to an n8n verification workflow that screens each image together with the listing text using a hosted multimodal large-language model.

FR-V2. The workflow shall return a decision (*approve* or *flag*) with a textual rationale, written back to the entity through a callback endpoint.

FR-V3. Flagged entities shall appear in the moderation queue for administrative review.

Featured Content Curation.

FR-C1. The Administrator shall mark any store or product as *featured*, promoting it on the landing page.

FR-C2. In the absence of featured products for a tag, the recommendation engine shall fall back to top-selling products of that tag.

2.1.3 Non-Functional Requirements

Beyond pure functionality, the deliverable must exhibit the qualities summarised in Table 2.1, prioritised with the professional supervisor during the requirements review.

Table 2.1: Non-functional requirements

Criterion	Description
Performance	Catalogue and store pages render in under one second on a 10 Mbit/s connection; typical API endpoints respond in under 300 ms.
Security	Passwords hashed by Better-Auth; sessions in HTTP-only, `SameSite=Lax` cookies; HTTP security headers via Helmet; uploads validated by type and size; every protected route gated by an auth guard and the `@Roles()` decorator, with object-level ownership enforced in the service layer. Required secrets are validated at start-up.
Scalability	The backend is stateless and horizontally scalable behind a reverse proxy; uploads live on a volume that can migrate to object storage with no code change.
Availability	The production stack runs under Docker Compose with restart policies; deployment is zero-downtime through rolling container replacement.
Usability	The frontend follows accessible component patterns, is fully responsive, and uses slug-based, human-readable URLs.
Maintainability	The codebase is split into bounded NestJS modules, the schema is versioned through Prisma migrations, and the infrastructure is reproducible through Terraform.
Observability	Application logs are emitted to `stdout` and aggregated by Docker; the chat assistant offers a queryable view of the database for diagnostics.

2.2 Global Use-Case Diagram

Figure 2.1 groups the principal use cases of the platform around its three actors. Two dotted dependencies illustrate the cross-actor flows on which the rest of the analysis relies: a seller application *triggers* an administrative review, and product creation is automatically *verified by AI*.

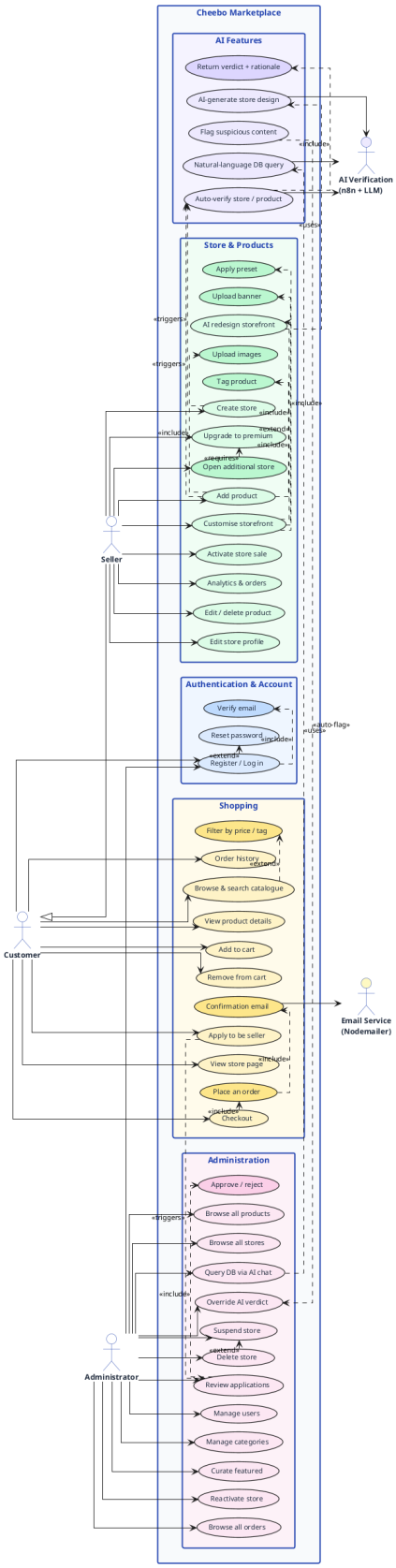

Figure 2.1: Global use-case diagram

2.3 Class Diagram

Figure 2.2 presents the static domain model. A single `User` entity carries the customer, seller and administrator roles; a Seller owns one or more `Store` entities, each holding `Product` entities that are organised by `Category` and `Tag`; orders are modelled by an `Order` aggregate and its `OrderItem` lines, allowing a single checkout to span several stores.

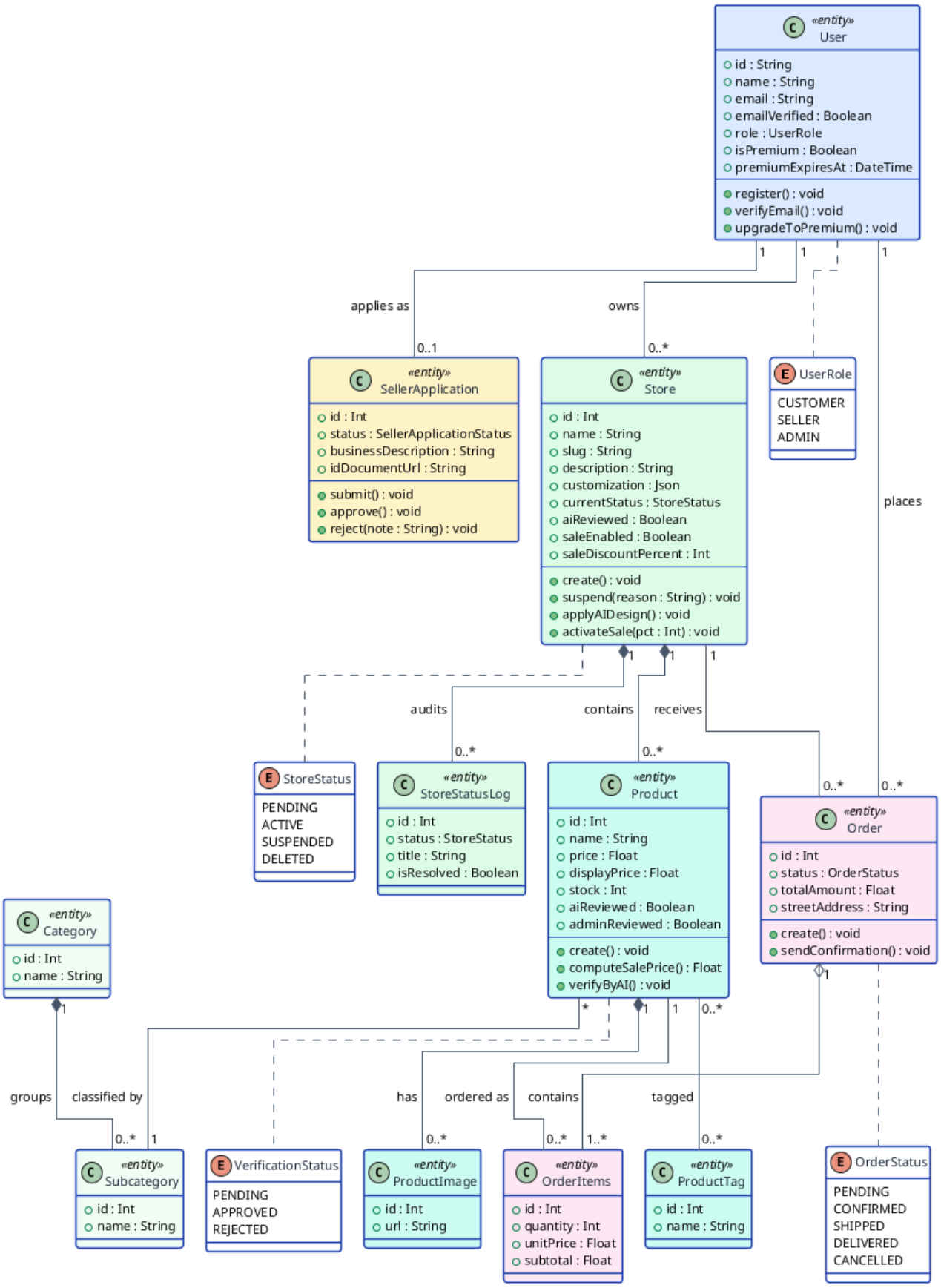

Figure 2.2: Class diagram of the platform domain model

2.4 Project Management with Scrum

2.4.1 Scrum Team

The Scrum accountabilities were distributed as follows:

- **Product Owner:** Mr. Mohammed Aziz Chibani (NeuralBey), who owned the product vision and prioritised the backlog.
- **Scrum Master and Development Team:** Nadim Jebali (the student), responsible for applying the framework and for the entire technical realisation.

2.4.2 MoSCoW Priority Legend

Each user story carries a **MoSCoW** priority and a **complexity** estimate, defined in Table 2.2.

Table 2.2: MoSCoW priority and complexity legend

MoSCoW Priority		
Must	*Must have*	Essential feature, required for the MVP.
Should	*Should have*	Important but not blocking.
Could	*Could have*	Desirable, included if time permits.
Won't	*Won't have*	Out of scope for this version.
Complexity		
1	Simple	Straightforward implementation, minimal effort.
2	Moderate	Average complexity, moderate effort.
3	Complex	Significant effort, elaborate logic.
5	Very complex	High effort, advanced technical challenge.

2.4.3 Product Backlog

Table 2.3 presents the product backlog as user stories following the template *"As a <role>, I want <action> so that <benefit>"*. The columns **P** and **Cx** indicate the MoSCoW priority and the estimated complexity.

Table 2.3: Product backlog — user stories with MoSCoW priority and complexity

Feature	ID	User Story	P	Cx
F1 Authentication	US01	As a visitor, I want to create an account with my email so that I can access the platform.	**Must**	**2**
	US02	As a visitor, I want a verification email so that I can activate my account securely.	**Must**	**2**
	US03	As a user, I want to sign in with my credentials so that I can access my dashboard.	**Must**	**1**
	US04	As a user, I want to sign out so that I can protect my account on a shared device.	**Must**	**1**
F2 Store Management	US05	As a seller, I want to create a store with a name, description and logo so that customers can recognise my brand.	**Must**	**2**
	US06	As a seller, I want my store URL to use a readable slug so that it is easy to share.	**Must**	**2**
	US07	As a seller, I want to customise my storefront across seven sections through a WYSIWYG editor so that I can style my store live.	**Should**	**5**
	US08	As a seller, I want to apply a preset so that I can get a professional look quickly.	**Should**	**2**
	US09	As a premium seller, I want to operate multiple stores so that I can segment my offerings.	**Should**	**2**
	US34	As a seller, I want a store-wide percentage sale so that all my products show discounted prices and a badge automatically.	**Should**	**2**
F3 Product Management	US10	As a seller, I want to create, edit and delete products so that I can keep my catalogue up to date.	**Must**	**2**

Continued on next page

Feature	ID	User Story	P	Cx
	US11	As a seller, I want to upload multiple product images so that customers can see my products clearly.	**Must**	**2**
	US12	As a seller, I want to assign tags to products so that the recommendation engine can surface them.	**Should**	**1**
	US13	As a customer, I want to browse products by category so that I can discover relevant items.	**Must**	**1**
F4 Order System	US14	As a customer, I want to add products to a cart so that I can purchase multiple items at once.	**Must**	**2**
	US15	As a customer, I want to check out across multiple stores in a single order so that I avoid separate transactions.	**Must**	**3**
	US16	As a customer, I want a confirmation email after ordering so that I have a record of my purchase.	**Must**	**1**
	US17	As a seller, I want to view orders containing my products so that I can fulfil them.	**Must**	**2**
F5 Administration	US18	As an admin, I want to review seller applications so that I can approve legitimate merchants.	**Must**	**2**
	US19	As an admin, I want to suspend or delete any store so that I can enforce marketplace rules.	**Must**	**2**
	US20	As an admin, I want to curate featured stores and products so that the landing page promotes quality.	**Should**	**1**
	US21	As an admin, I want to manage categories so that the catalogue stays organised.	**Must**	**1**
F6 AI Verification	US22	As the system, I want to verify every new store and product via an LLM so that low-quality content is flagged automatically.	**Should**	**3**

Continued on next page

Feature	ID	User Story	P	Cx
	US23	As an admin, I want to override any AI verdict so that I remain the final authority.	**Must**	**1**
	US24	As a customer, I want an AI-verified badge on trusted products so that I can shop with confidence.	**Should**	**1**
	US35	As a seller, I want to regenerate my storefront from a natural-language prompt so that I get a professional design without manual work.	**Could**	**3**
F7 Premium Tier	US25	As a seller, I want to subscribe to a premium plan so that I can unlock advanced features.	**Should**	**3**
	US26	As a premium seller, I want all customisation controls so that I can fully personalise my store.	**Should**	**3**
	US27	As the system, I want to preserve premium customisation after expiry so that the store keeps its design.	**Could**	**2**
F8 CI/CD & Deployment	US28	As a developer, I want every service in a Docker container so that environments are identical.	**Must**	**2**
	US29	As a developer, I want images built and pushed automatically on every push to main so that deployments are current.	**Must**	**2**
	US30	As a developer, I want the droplet provisioned via Terraform so that infrastructure is reproducible.	**Should**	**3**
	US31	As a developer, I want deployment over SSH on every push to main so that updates reach production automatically.	**Must**	**3**
	US32	As a developer, I want secrets managed through CI secrets so that no credentials live in the repository.	**Must**	**1**

Continued on next page

Feature	ID	User Story	P	Cx
	US33	As a developer, I want DNS records updated automatically to the new droplet IP so that the application is always reachable.	**Should**	**2**

2.4.4 Analysis Summary

The product backlog comprises **35 user stories** across **8 features**. The bulk are *Must* or *Should* priorities forming the marketplace core, while a few advanced capabilities (AI redesign, premium-customisation preservation) are *Could* items scheduled only once the essentials are delivered. Complexity estimates concentrate effort on the storefront editor, the multi-store checkout, the AI verification pipeline and the deployment automation.

2.4.5 Sprint Planning per Release

The backlog was organised into **two releases of two sprints each**. The first release delivers the social and transactional foundations; the second adds intelligence, monetisation and the deployment pipeline. Table 2.4 maps each sprint to its goal and user stories.

Table 2.4: Sprint planning across the two releases

Release	Sprint	Goal	User stories
Release 1	Sprint 1	Authentication and seller onboarding	US01–US04, US18
	Sprint 2	Store and product management	US05–US13, US34
Release 2	Sprint 3	Orders and administration	US14–US17, US19–US21
	Sprint 4	AI verification, premium tier and DevOps	US22–US33, US35

2.5 Gantt Diagram

Figure 2.3 presents the project schedule over the internship: an inception phase, the four sprints grouped into two releases with their delivery milestones, a tests-and-deployment phase, and a report-writing activity running in parallel throughout.

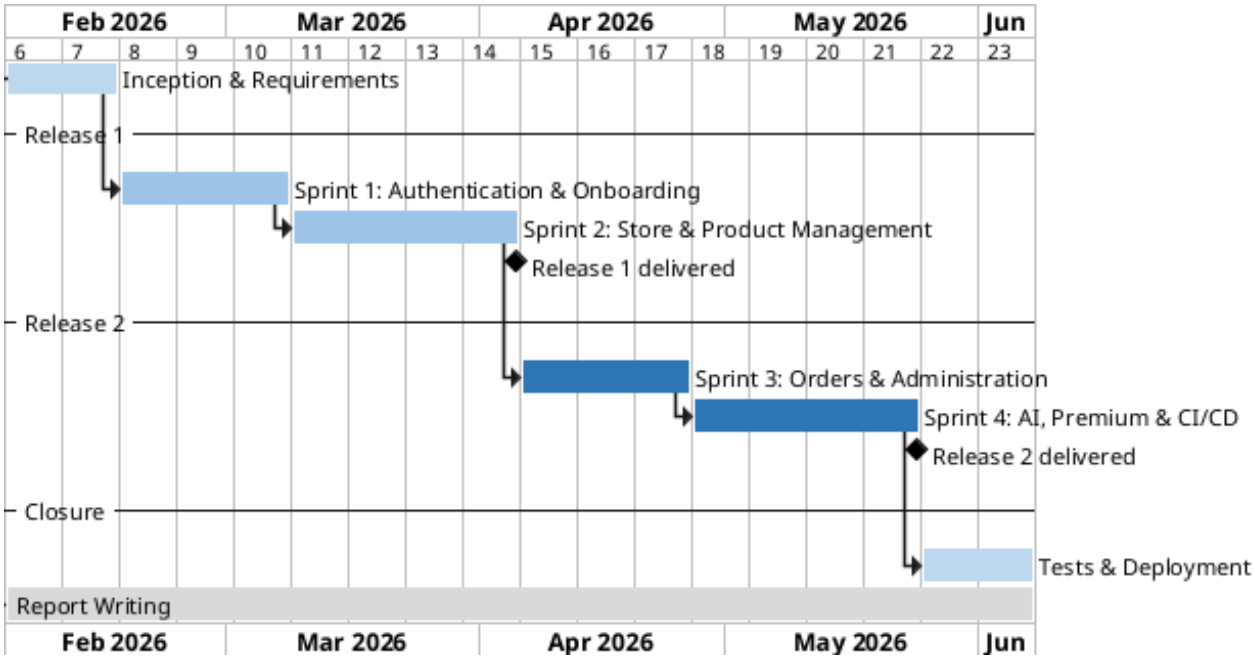

Figure 2.3: Project Gantt chart (two releases, four sprints)

2.6 Architecture and Technical Choices

2.6.1 Logical Architecture

The platform follows a classic **three-tier architecture** that separates the user interface (presentation), the business logic (application) and the persistent data (data) into independent layers, with an **n8n** automation layer orbiting the backend to host the AI workflows. End users reach the platform over HTTPS; a reverse proxy serves the React bundle and forwards `/api/*` requests to the NestJS API, which persists state to MariaDB through Prisma and exchanges webhooks with n8n. Figure 2.4 shows the topology.

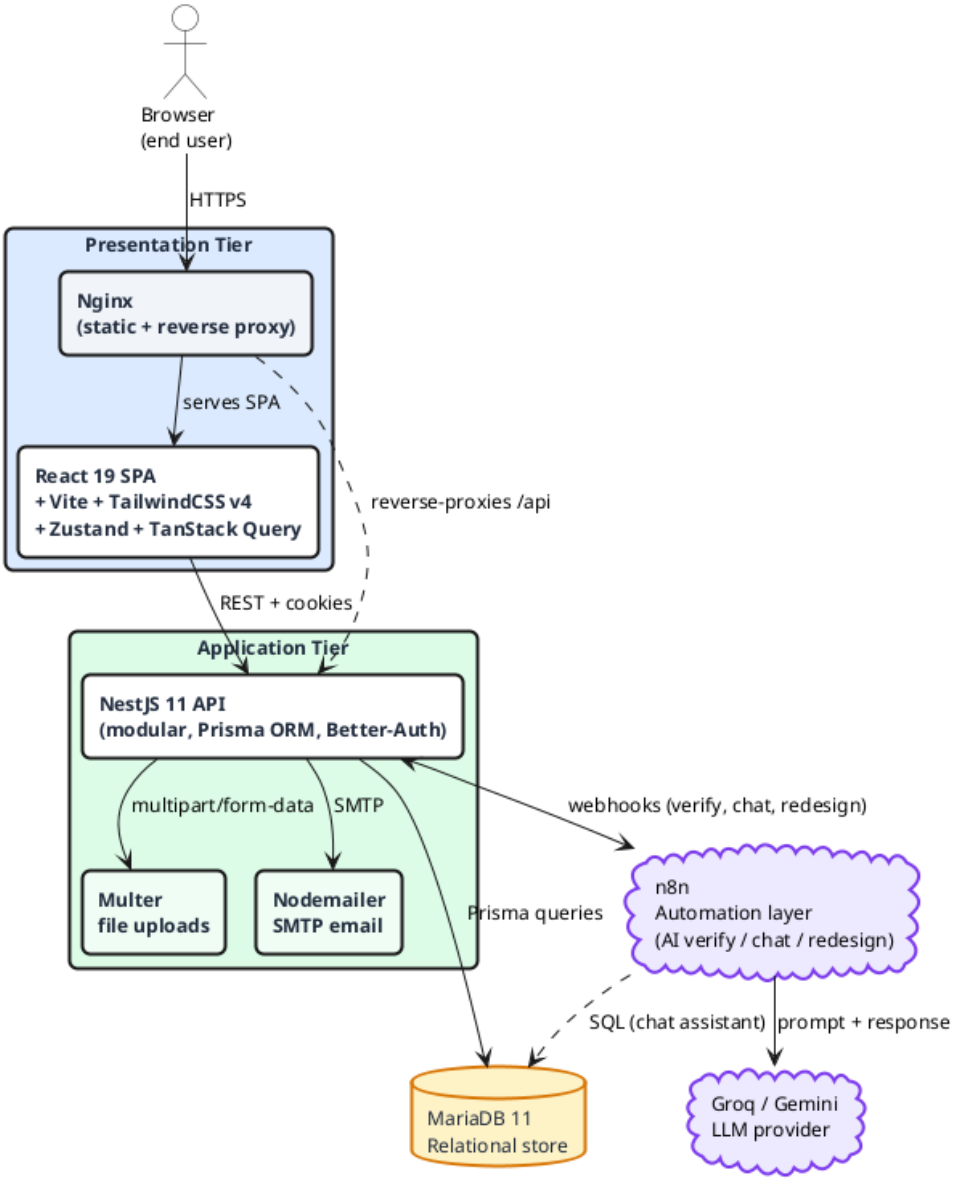

Figure 2.4: Logical architecture (three tiers and the n8n automation layer)

2.6.2 Physical Architecture

In production, every component runs as a **Docker** container orchestrated by Docker Compose on a single DigitalOcean droplet. A **Caddy** reverse proxy terminates TLS with automatically renewed Let's Encrypt certificates and routes traffic to the frontend, backend and n8n containers over an internal bridge network; uploads and the n8n state persist on Docker volumes. Figure 2.5 details the container topology.

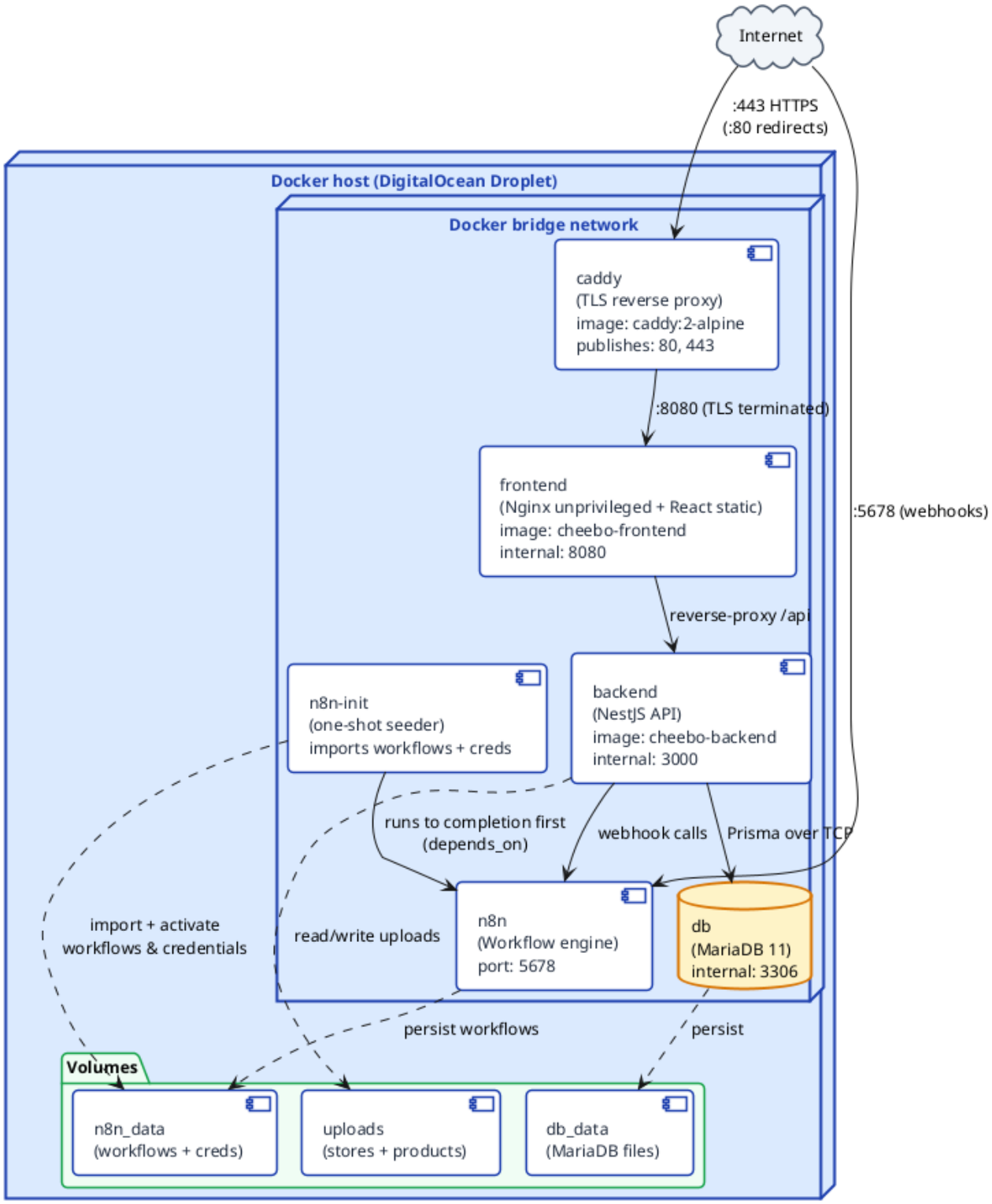

Figure 2.5: Physical architecture (Docker Compose services)

2.6.3 Artificial-Intelligence Models

The intelligent features are powered by hosted large-language models, orchestrated through n8n so that providers can be swapped without touching the backend. Table 2.5 lists the model used by each service.

Table 2.5: AI models used per service

Service	Model / provider
Store and product verification	Multimodal LLM (Google Gemini) screening images and listing text
AI storefront redesign	Groq `llama-3.3-70b-versatile` generating the storefront customisation
Database chat assistant	Hosted LLM translating natural-language questions into safe database queries

2.6.4 Tools, Frameworks and Languages

The system is implemented in **TypeScript** end to end. Table 2.6 lists the tools used throughout the project — covering source control, project tracking, development, containerisation, infrastructure provisioning, workflow automation and API testing — while Table 2.7 summarises the main frameworks and libraries.

Table 2.6: Tools used throughout the project

Logo	Tool	Role
	GitHub	Version control, pull-request review and CI/CD via GitHub Actions.
	Notion	Backlog management, task board and record of user stories.
	VS Code	Primary IDE, extended for TypeScript, Prisma, ESLint and Git.
	Docker	Containerisation of all services for dev/prod parity.
	Terraform	Infrastructure-as-code provisioning of the DigitalOcean resources.
	n8n	Visual workflow automation orchestrating the AI features.
	Postman	Manual testing and documentation of the REST endpoints.
	pnpm	Fast, disk-efficient package manager for the monorepo.

Table 2.7: Frameworks and languages

Technology	Role
TypeScript	Statically-typed language used across the frontend and the backend.
React	Single-page application library for the customer, seller and admin interfaces.
NestJS	Modular Node.js framework structuring the backend into feature modules.
Prisma	ORM and migration tool over MariaDB.
Better-Auth	Authentication library handling sessions and email verification.
Docker	Container runtime packaging every service.
Terraform	Declarative provisioning of the cloud infrastructure.
n8n	Workflow engine hosting the AI orchestration.

2.6.5 Database

The platform stores its data in a **MariaDB** relational database accessed through the **Prisma** ORM, which provides a type-safe client and versioned migrations. The database schema follows directly from the domain model: the entities and associations of the class diagram presented earlier (Figure 2.2) map onto the database tables — users and their roles, stores and their customisation, products with categories and tags, and orders with their line items — with Prisma generating and versioning the corresponding migrations.

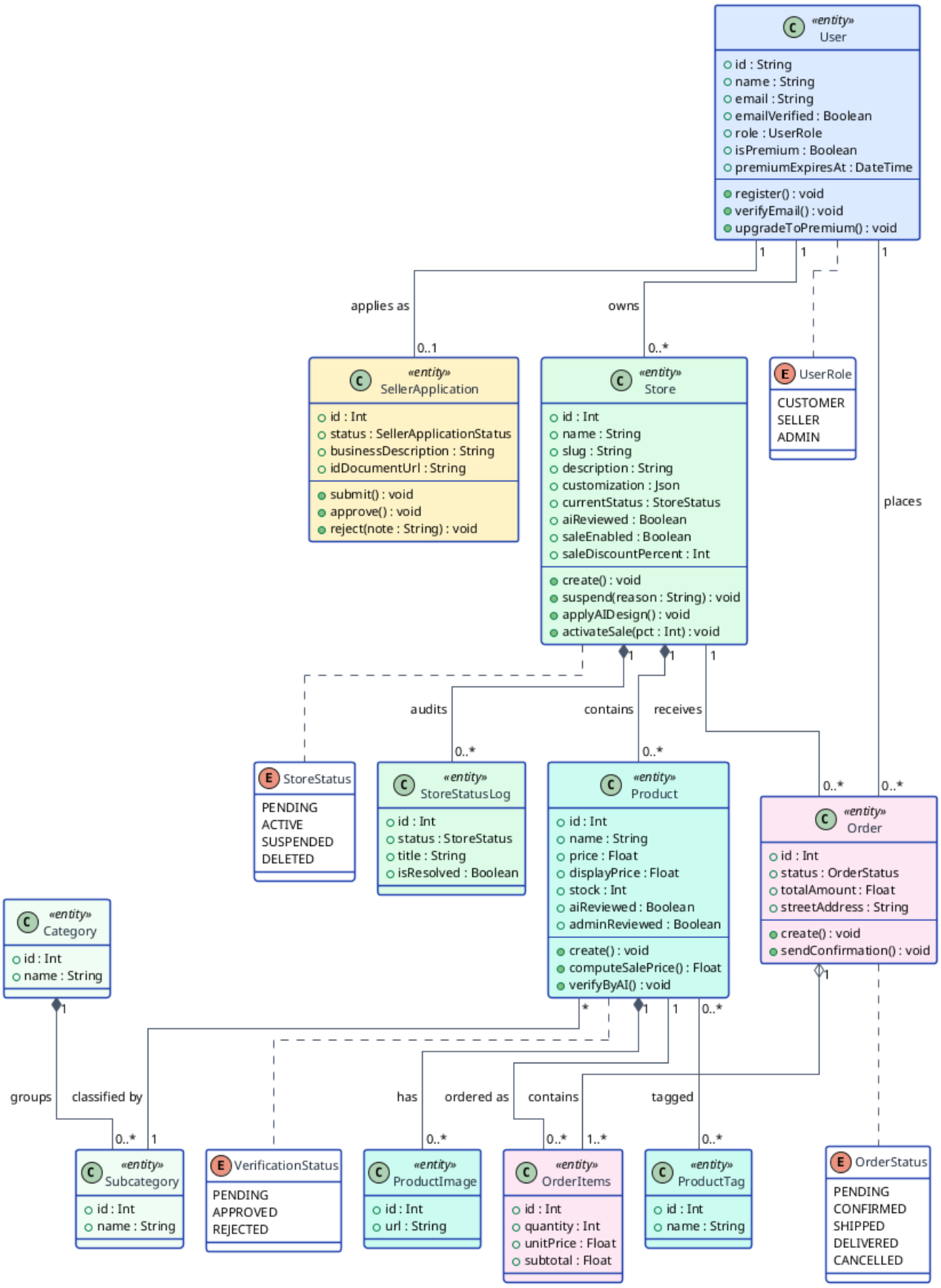

Figure 2.6: Class diagram underpinning the database schema

Conclusion

This chapter formalised the requirements of the platform. The three actors were identified, the functional requirements grouped into seven domains and the non-functional requirements stated, then illustrated with the global use-case diagram and the class diagram. The Scrum management of the project was presented — the team, the MoSCoW-prioritised product backlog of 35 user stories across 8 features, and the planning of four sprints over two releases — together with the schedule and the architecture and technical choices that frame the realisation. The following chapters describe the development of the two releases.

Conclusion générale

Bibliographie

[B1] Pierre Pezziardi, Référentiel des Pratiques Agiles, édition ebook.2013

Webographie

[1] K. Beck et al. “Manifesto for agile software development.” [Accessed 2026]. [Online]. Available: https://agilemanifesto.org/

[2] K. Schwaber and J. Sutherland. “The scrum guide: the definitive guide to scrum.” [Accessed 2026]. [Online]. Available: https://scrumguides.org/

[3] Object Management Group. “Unified modeling language (uml) specification, version 2.5.1.” [Accessed 2026]. [Online]. Available: https://www.omg.org/spec/UML/

Résumé

Ce rapport présente la conception et le développement d'une place de marché électronique multi-vendeurs, réalisé dans le cadre d'un projet de fin d'études au sein de la société NeuralBey. La plateforme permet à des vendeurs indépendants d'ouvrir leurs propres boutiques, de publier des produits et de traiter des commandes au sein d'un environnement partagé et gouverné de façon centralisée, tandis que les administrateurs valident les vendeurs, modèrent le contenu et mettent en avant une sélection de qualité. Sa principale particularité est un pipeline de vérification assisté par l'intelligence artificielle qui contrôle automatiquement les boutiques et les produits afin de préserver la confiance des acheteurs à grande échelle, complété par un assistant conversationnel de base de données et une refonte de vitrine assistée par IA. Le système est bâti comme une application React monopage adossée à une API modulaire NestJS et une base de données MariaDB, n8n orchestrant les traitements d'intelligence artificielle, et il est livré au moyen d'une chaîne DevOps entièrement conteneurisée et décrite comme code (Docker, Terraform, GitHub Actions). Le projet a suivi une démarche agile Scrum répartie sur deux releases.

Mots clés : Place de marché multi-vendeurs, E-commerce, Intelligence Artificielle, NestJS, React, Scrum, DevOps

Abstract

This report presents the design and development of a multi-vendor e-commerce marketplace, carried out as a final-year engineering project at NeuralBey. The platform lets independent sellers open their own stores, publish products and fulfil orders within a shared, centrally-governed environment, while administrators onboard sellers, moderate content and curate featured selections. Its distinguishing feature is an AI-assisted verification pipeline that automatically screens stores and products to protect buyer trust at scale, complemented by a conversational database assistant and AI-assisted storefront redesign. The system is built as a React single-page application backed by a modular NestJS API and a MariaDB database, with n8n orchestrating the AI workflows, and is delivered through a fully containerised, infrastructure-as-code DevOps pipeline (Docker, Terraform, GitHub Actions). The project followed an agile Scrum approach spread over two releases.

Keywords : Multi-vendor marketplace, E-commerce, Artificial Intelligence, NestJS, React, Scrum, DevOps